Project Proposal:
Stack Overflow Content Recommendation System

MSDA 3999: Capstone Practicum

Field Project

Author: Tarun Kumar Jasti, Satyaki Mitra, Sharmendra Desiboyina

Advisor: Prof. Khalid Aboalayon

Master's in Data Analytics

Clark University
03.18.2026

TABLE OF CONTENTS

TABLE OF CONTENTS 2
DECLARATION 3
ABSTRACT 4
CHAPTER ONE INTRODUCTION 5
1.1 Background 5
1.2 Aim and objectives 5
1.3 Research Question / Hypotheses 5
1.4 Research Motivation 6
1.5 Research contribution 6
CHAPTER TWO LITERATURE REVIEW 7
CHAPTER THREE RESEARCH METHODOLOGY 11
3.1 Research approach 11
3.2 Techniques and Procedure 11
3.2.1 Dataset Description 11
3.2.2 Data Collection 11
3.2.3 Data Processing 12
3.2.4 Expected Outcome 13
3.2.5 Evaluation 13
3.3 Timeline 14
REFERENCES 18
APPENDIX A – ..... **Error! Bookmark not defined.**

DECLARATION

I(We) hereby declare that this project entitled Stack Overflow Content Recommendation System' is entirely my(our) own work and it has never been submitted nor is it currently being submitted for any other degree.

Signed: Satyaki Mitra, Sharmendra Desiboyina, Tarun Kumar Jasti
Date: 03.20.2026

ABSTRACT

"This project focuses on building a Stack Overflow content recommendation system that helps developers efficiently discover relevant programming solutions. Traditional keyword-based search fails to capture semantic similarity between queries, leading to irrelevant results and reduced productivity.

To address this, the system combines semantic embeddings using transformer-based models with quality-based ranking signals such as votes, views, and accepted answers. The dataset is extracted from the Stack Overflow BigQuery public dataset, consisting of approximately 500,000 questions and top-ranked answers.

The system uses Sentence-BERT embeddings, FAISS vector search, and re-ranking techniques to deliver highly relevant recommendations. Additionally, personalization is incorporated by modeling user interaction history and preferences.

The expected outcome is a scalable recommendation system capable of delivering real-time, high-quality, and personalized results, demonstrating an end-to-end machine learning pipeline aligned with industry practices."

Keywords: Semantic Search; FAISS; NLP; Machine Learning; LLM; Vector Search; Recommendation System

CHAPTER ONE: INTRODUCTION

1.1 Background

1. "Stack Overflow hosts over 24 million questions, making it one of the largest knowledge-sharing platforms for developers. However, due to the scale and diversity of content, developers often struggle to find relevant solutions efficiently.
2. Traditional search systems rely on keyword matching, which fails to capture semantic meaning. For example, queries like *'sort array'* and *'order elements in list'* represent the same intent but may not retrieve similar results.
3. Modern recommendation systems used by platforms such as Google, Netflix, and Amazon combine semantic understanding with ranking signals to improve relevance.
4. This project applies similar principles to build a hybrid recommendation system that integrates semantic similarity with content quality and user preferences."

1.2 Aim and objectives

Aim:

"To develop an end-to-end Stack Overflow recommendation system that provides relevant, high-quality, and personalized programming content."

Objectives:

- Extract and preprocess large-scale Stack Overflow data
- Generate semantic embeddings using transformer models
- Build a fast retrieval system using FAISS
- Incorporate ranking signals such as votes and accepted answers
- Develop personalization using user interaction data
- Achieve efficient retrieval with low latency (<100ms)

1.3 Research Question / Hypotheses

"Given a user's technical interests and interaction history, how can we recommend the most relevant Stack Overflow questions by combining semantic similarity, content quality, and personalization signals?

1.4 Research Motivation

"This project is motivated by the growing importance of recommendation systems in modern technology platforms. Companies such as Google, Netflix, and LinkedIn rely heavily on recommendation systems to improve user engagement.

Additionally, improving developer productivity has a significant impact on software development efficiency. A well-designed recommendation system can reduce time spent searching for solutions and expose developers to better practices.

From a career perspective, this project aligns with roles in Machine Learning Engineering and Cloud Systems, focusing on building scalable, production-ready systems."

1.5 Contribution

"This project contributes to the field by:

- Demonstrating a hybrid recommendation system combining semantic embeddings and ranking signals
- Providing an end-to-end ML pipeline from data extraction to deployment
- Implementing scalable techniques such as FAISS vector search and embedding caching
- Incorporating personalization using user interaction data
- Delivering a practical system aligned with industry best practices

The project also serves as a portfolio-ready implementation for real-world machine learning systems."

CHAPTER TWO: LITERATURE REVIEW

2.1 Overview

"Recommendation systems and semantic search techniques have become essential in modern information retrieval systems. Traditional keyword-based approaches fail to capture contextual meaning, leading to irrelevant results. Recent research focuses on leveraging transformer-based models, deep learning architectures, and efficient similarity search techniques to improve recommendation quality.

This section reviews key studies related to semantic embeddings, recommendation systems, and Stack Overflow-based research, and explains how they influence the design of this project."

2.2 Semantic Embeddings for Recommendation Systems

A study titled *"Improving Code Example Recommendations Using BERT and Query-Aware LSH"* demonstrates the effectiveness of BERT-based embeddings for capturing semantic meaning in Stack Overflow content. The authors convert text into vector representations and apply similarity search techniques to retrieve relevant results.

Their findings show that semantic embeddings significantly outperform traditional methods such as TF-IDF, improving recommendation accuracy and relevance by up to 20–35%.

However, the study relies on LSH-based approximate search, which may compromise accuracy for speed.

Relevance to our project:
This work validates the use of transformer-based embeddings. Our system extends this by using FAISS for efficient and more accurate vector search, along with ranking signals.

2.3 Deep Learning for Quality Assessment

Another study focuses on evaluating question quality using deep learning models such as LSTM, GRU, and BERT. The results show that transformer-based models achieve the highest accuracy (~93%), outperforming traditional neural networks.

The study highlights the importance of content quality evaluation in improving user

Limitation:
This work focuses only on classification of quality and does not address recommendation or personalization.

Relevance to our project:
Our system incorporates quality signals (votes, accepted answers, views) to rank recommendations, combining semantic relevance with content quality.

2.4 Deep Learning for Tag Prediction

The paper *"Predicting Tags for Learner Questions on Stack Overflow"* proposes a deep learning architecture combining:

- BERT (semantic understanding)
- CNN (local pattern extraction)
- BiLSTM (sequence learning)
- Attention mechanism (important word focus)

The model achieves high performance with ~89% precision and recall, demonstrating the effectiveness of combining multiple deep learning techniques.

Limitation:
The study focuses on tag prediction, not recommendation or retrieval.

Relevance to our project:
This paper supports the importance of semantic understanding and contextual modeling, which we apply for question similarity and recommendation.

2.5 Limitations of Existing Work

From the reviewed literature, several limitations are identified:

- Many systems focus only on semantic similarity, ignoring content quality
- Some approaches use approximate methods (LSH), reducing accuracy
- Lack of personalization based on user behavior
- Limited focus on end-to-end production-level systems

2.6 Research Gap

Based on the analysis, there is a clear gap in developing a system that:

- Combines semantic embeddings + ranking signals + personalization
- Uses efficient yet accurate retrieval (FAISS instead of LSH)
- Implements a complete pipeline from data processing to deployment
- Supports real-time recommendations at scale

2.7 Contribution of This Project

"This project addresses the identified gaps by building a hybrid recommendation system that integrates:

- Transformer-based embeddings for semantic understanding
- FAISS for fast and accurate vector retrieval
- Cross-encoder for improved ranking
- Quality-based scoring using votes, views, and accepted answers
- Personalization using user interaction history

Unlike existing work, this system provides a complete, scalable, and production-ready solution for Stack Overflow content recommendation.”

CHAPTER THREE RESEARCH METHODOLOGY

3.1 Research approach

"This project follows a data-driven machine learning approach to build a scalable recommendation system for Stack Overflow content.

The methodology combines:

- Natural Language Processing (NLP) for semantic understanding
- Vector similarity search for retrieval
- Ranking algorithms for quality-based ordering
- User modeling techniques for personalization

The system is designed as an end-to-end pipeline, starting from data extraction and preprocessing, followed by embedding generation, efficient retrieval using FAISS, and final re-ranking using multiple signals.

This approach aligns with modern industry systems where semantic understanding is combined with ranking and personalization to improve recommendation quality."

3.2 Techniques and Procedure

3.2.1 Dataset Description

"The dataset is sourced from the Stack Overflow public dataset available on Google BigQuery.

After processing, the dataset includes:

- ~500,000 questions
- ~1.5 million answers (top 3 answers per question)
- 15+ engineered features

Key attributes include:

- Question title, body, and tags
- Score, view count, answer count
- Accepted answer indicator
- Derived features such as:
    - body_word_count
    - title_length
    - has_code
    - num_tags

This dataset provides sufficient scale and diversity to build a robust recommendation system."

3.2.2 Data Collection

"Data was collected using an optimized SQL query on Google BigQuery.

Key steps include:

- Extraction from `posts_questions` and `posts_answers` tables
- Use of Common Table Expressions (CTEs) and ROW_NUMBER() for ranking answers
- Selection of top 30 programming tags
- Application of stratified sampling to ensure balanced representation
- Retrieval of top 3 answers per question based on score

The query was optimized to scan approximately 35GB of data, avoiding repeated scans and ensuring efficient data collection."

3.2.3 Data Processing

"The data processing pipeline consists of multiple stages:

1. Data Cleaning

- Removal of HTML using BeautifulSoup
- Extraction of code blocks

- Normalization of text and whitespace
- Removal of noise such as URLs

2. Feature Engineering

- Computation of:
    - body_word_count
    - title_length
    - has_code
    - has_accepted_answer
    - num_tags

3. Text Preparation for Embedding

- Combination of **title, tags, and body** into a single text field
- Structured format: Title + Tags + Body
- Ensures the embedding model captures **full semantic context**

4. Embedding Generation

- Model used: **Sentence-BERT (all-MiniLM-L6-v2)**
- Each question converted into a **384-dimensional vector**
- Enables semantic similarity comparison between questions

5. Vector Storage

- Embeddings stored as a matrix: (498644 × 384)
- Saved for efficient retrieval

This pipeline transforms raw Stack Overflow data into a **semantic representation suitable for recommendation tasks**.

3.2.4 Retrieval System (FAISS)

"To enable fast similarity search, the system uses **FAISS** (Facebook AI Similarity Search).

- Stores all question embeddings
- Performs nearest neighbor search using cosine similarity
- Retrieves top-K similar questions for a given query

A **Flat Index** is used because:

- Dataset size (~500K vectors) allows fast exact search (~milliseconds)
- Avoids accuracy loss associated with approximate methods

This ensures high accuracy and low latency retrieval."

3.2.5 Ranking and Re-Ranking

"Retrieval alone is not sufficient for high-quality recommendations. Therefore, a **re-ranking layer** is applied.

1. Cross-Encoder Re-ranking

- Model: `cross-encoder/ms-marco-MiniLM-L-6-v2`
- Evaluates query and candidate together
- Captures deeper contextual relationships

2. Quality-Based Scoring

- Final ranking combines multiple factors:
- Semantic similarity score
- Question score (votes)
- View count (popularity)
- Accepted answer presence
- Tag relevance

Final ranking formula:

Final Score = weighted combination of:

semantic_score + quality_score + tag_overlap + user_interest

This ensures that results are not only relevant but also **high quality**."

3.2.6 Personalization

"The system incorporates personalization using **user interaction data**.

As shown in the project workflow:

- User clicks and searches are logged
- Tag preferences are updated dynamically
- User interest embedding is computed as:

```
average of embeddings of clicked questions
```

On subsequent interactions:

- System retrieves questions similar to user interests
- Combines:
  - semantic similarity
  - tag preferences
  - interaction history

This enables **personalized recommendations instead of generic search results**."

3.2.7 Expected Outcome

"The final system is expected to:

- Provide semantic search results instead of keyword matching
- Deliver high-quality ranked recommendations
- Support personalized content discovery
- Achieve low-latency retrieval (<100ms)
- Demonstrate a complete end-to-end ML pipeline

This system reflects a production-level recommendation architecture similar to modern industry systems."

3.2.8 Evaluation

"This project will be evaluated using both retrieval performance metrics and system efficiency metrics.

1. Retrieval Evaluation Metrics

- Precision@K
  Measures how many of the top-K recommended questions are relevant.

- Recall@K
  Measures how many relevant questions are successfully retrieved.
- Mean Reciprocal Rank (MRR)
  Evaluates how early the first relevant result appears in the ranked list.
- Hit Rate@K
  Checks whether at least one relevant result appears in the top-K results.

These metrics are commonly used in recommendation systems and will help assess accuracy and relevance.

2. Ranking Evaluation

To evaluate ranking quality, we consider:

- Semantic similarity score
- Quality signals (votes, views, accepted answers)
- User preference alignment

The effectiveness of re-ranking will be analyzed by comparing:

- Results before re-ranking (FAISS output)
- Results after re-ranking (Cross-Encoder + scoring)

3. System Performance Metrics

- **Latency**: Target <100ms response time
- **Scalability**: Ability to handle large datasets (~500K questions)
- **Efficiency**: Speed of FAISS retrieval and embedding lookup

4. User-Level Evaluation (Optional Extension)

If implemented, user interactions (clicks, searches) will be used to measure:

- Engagement improvement
- Personalization effectiveness

3.3 Timeline

Phase	Task	Duration
Phase 1	Problem Definition & Literature Review	Week 1–2
Phase 2	Data Extraction (BigQuery)	Week 3
Phase 3	Data Cleaning & Feature Engineering	Week 4–5
Phase 4	Exploratory Data Analysis (EDA)	Week 6
Phase 5	Embedding Generation (Sentence-BERT)	Week 7
Phase 6	FAISS Index Construction	Week 8
Phase 7	Re-ranking & Quality Scoring	Week 9
Phase 8	Personalization Module	Week 10
Phase 9	Evaluation & Testing	Week 11
Phase 10	Deployment / Final Report	Week 12

REFERENCES

- Rahmani, S., Naghshzan, A., & Guerrouj, L. (2023). *Improving Code Example Recommendations Using BERT and Query-Aware LSH*. arXiv.
- IEEE (2025). *Deep Learning Models for Quality Assessment of Stack Overflow Questions*.
- Springer (2024). *Predicting Tags for Learner Questions on Stack Overflow*.
- Stack Overflow Public Dataset. Google BigQuery.